

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ TACCHI
Nom d'usage ▶
Prénom ▶ Antonin
Adresse ▶ 3 allée ambroise croizat 13110 Port-de-Bouc

Titre professionnel visé

Développeur Web et Web Mobile (DWWM)

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.
Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile sécurisée p. 5

- ▶ Installer et configurer son environnement de travail en fonction du projet web ou web mobile.....p.....5
- ▶ Maquetter des interfaces utilisateur web ou web mobile.....p.....8....
- ▶ Réaliser des interfaces utilisateur statiques web ou web mobile.....p.....11....
- ▶ Développer la partie dynamique des interfaces utilisateur web ou web mobile.....p.....14....

Développer la partie back-end d'une application web ou web mobile sécurisée p.

- ▶ Mettre en place une base de données relationnelle.....p.....17...
- ▶ Développer des composants d'accès aux données SQL et NoSQL.....p.....22...
- ▶ Développer des composants métier coté serveur.....p.....25....
- ▶ Documenter le déploiement d'une application dynamique web ou web mobile.....p.....

Titres, diplômes, CQP, attestations de formation (facultatif)

p. _____

Déclaration sur l'honneur

p. _____

Documents illustrant la pratique professionnelle (facultatif)

p. _____

Annexes (Si le RC le prévoit)

p. _____

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type

1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 ►

Installer et configurer son environnement de travail en fonction du projet web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

En début de formation à LaPlateforme, j'ai installé et configuré l'ensemble de mon environnement de développement web. Cette étape fondamentale m'a permis d'être opérationnel rapidement et de travailler dans les mêmes conditions que mes collègues.

WAMP (Windows Apache MySQL PHP) :

J'ai installé WAMP sur ma machine Windows afin de disposer d'un serveur local Apache et d'une base de données MySQL en local. WAMP permet de simuler un environnement serveur directement sur son poste de travail, ce qui est utile pour tester des applications web sans nécessiter de connexion à un serveur distant.

Visual Studio Code :

J'ai installé VS Code comme éditeur de code principal. VS Code est un éditeur léger, puissant et très extensible grâce à ses extensions. J'ai installé les extensions suivantes pour optimiser mon flux de travail :

- Prettier – formatage automatique du code
- ESLint – détection des erreurs JavaScript en temps réel
- Docker – gestion des conteneurs directement depuis l'éditeur
- GitLens – visualisation de l'historique Git ligne par ligne
- Thunder Client – test d'API REST directement dans VS Code

Docker Desktop :

J'ai installé Docker Desktop pour afficher les conteneurs Docker sur ma machine. Docker Desktop fournit une interface graphique pour visualiser, démarrer et arrêter les conteneurs, images et volumes. J'ai utilisé la documentation officielle Docker pour suivre la procédure d'installation adaptée à mon système d'exploitation. En installant Docker Desktop j'ai aussi installé Docker Engine qui lui me sert à créer, gérer les conteneurs et les images.

Install Docker Engine

This section describes how to install Docker Engine on Linux, also known as Docker CE. Docker Engine is also available for Windows, macOS, and Linux, through Docker Desktop. For instructions on how to install Docker Desktop, see: [Overview of Docker Desktop](#).

Installation procedures for supported platforms

Click on a platform's link to view the relevant installation procedure.

Platform	x86_64 / amd64	arm64 / aarch64	arm (32-bit)	ppc64le	s390x
CentOS	✓	✓		✓	
Debian	✓	✓	✓	✓	
Fedora	✓	✓		✓	
Raspberry Pi OS (32-bit)			⚠		
RHEL	✓	✓			✓
Ubuntu	✓	✓	✓	✓	✓
Binaries	✓	✓	✓		

MySQL via DockerHub :

Plutôt que d'installer MySQL directement sur la machine, j'ai récupéré l'image officielle mysql:8.0 depuis DockerHub. DockerHub est un registre public d'images Docker, qui permet de télécharger des environnements préconfigurés en une seule commande (docker pull mysql:8.0). Cela garantit une version identique de MySQL pour tous les membres de l'équipe.

GitHub Desktop :

J'ai installé GitHub Desktop pour faciliter la gestion du versionnage avec Git. GitHub Desktop propose une interface graphique intuitive pour effectuer des commits, créer des branches, fusionner du code et synchroniser avec le dépôt distant, sans avoir à mémoriser toutes les commandes Git en ligne de commande.

2. Précisez les moyens utilisés :

WAMP Server (serveur local Windows), Visual Studio Code avec extensions (Prettier, ESLint, Docker, GitLens, Thunder Client), Docker Desktop pour la gestion des conteneurs, DockerHub pour récupérer l'image MySQL 8.0, GitHub Desktop pour le versionnage, documentation officielle Docker (docs.docker.com) pour l'installation.

3. Avec qui avez-vous travaillé ?

Configuration réalisée individuellement en début de formation à LaPlateforme. Les choix d'outils étaient communs à toute la promotion afin de garantir un environnement homogène entre les apprenants.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ▶ LaPlateforme (centre de formation)

Chantier, atelier, service ▶ Mise en place de l'environnement de développement

Période d'exercice ▶ Du 16/06/2025 au 18/06/2025

5. Informations complémentaires (facultatif)

Activité-type

1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶

Maquetter des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

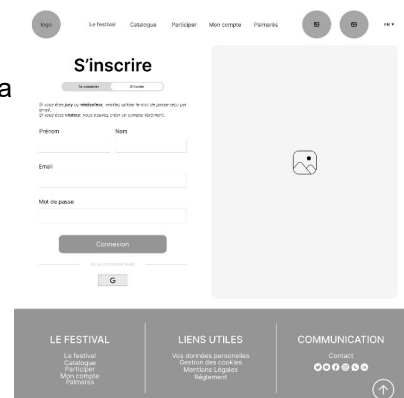
Dans le cadre du projet MarsAI (plateforme de courts-métrages), j'ai réalisé les maquettes complètes des interfaces utilisateur sur Figma avant de passer au développement. J'ai travaillé sur différents types de livrables, chacun correspondant à une étape du processus de conception :

Zoning :

Le zoning est la première étape de la conception. Il s'agit de découper la page en grandes zones fonctionnelles (header, contenu principal, sidebar, footer) sans se préoccuper du détail visuel. Cela permet de définir rapidement la structure et la hiérarchie de l'information.

Wireframe :

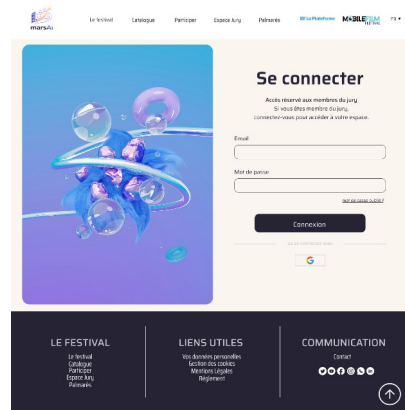
Le wireframe est une maquette basse fidélité en noir et blanc qui



représente la structure de l'interface avec ses éléments (boutons, champs, images) sans couleurs ni typographies définitives. J'en ai réalisé un pour la page d'inscription de MarsAI, ce qui a permis à l'équipe de valider l'architecture de la page avant d'investir du temps dans le design.

Mockup (maquette haute fidélité) :

Le mockup est la version finalisée de la maquette, avec les vraies couleurs, typographies, images et espacements. J'ai réalisé le mockup de la page de connexion de MarsAI sur Figma, en respectant la charte graphique du projet (tons sombres, dégradés bleutés, logo MarsAI).



Prototype :

Le prototype est une maquette interactive qui simule la navigation entre les écrans. Sous Figma, j'ai lié les boutons et les liens entre les différentes pages pour que l'équipe puisse tester le parcours utilisateur avant le développement.

Composants Figma :

J'ai utilisé les composants Figma pour créer des éléments réutilisables dans toutes les maquettes : boutons, champs de formulaire, cards de courts-métrages, navigation. L'avantage des composants est que toute modification d'un composant de base se répercute automatiquement sur tous ses usages dans le projet, ce qui garantit la cohérence visuelle.

2. Précisez les moyens utilisés :

Figma (outil de conception collaborative en ligne) pour la création des wireframes, mockups et prototypes. Utilisation des composants Figma pour garantir la cohérence des éléments d'interface sur l'ensemble des écrans.

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. Les maquettes ont été réalisées collaborativement sur Figma, ce qui a permis à tous les membres de l'équipe de commenter et valider les choix de design avant le développement.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Projet MarsAI – Conception des interfaces

Période d'exercice ► Du 12/01/2026 au 16/01/2026

5. Informations complémentaires (facultatif)

Activité-type

1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°3 ▶

Réaliser des interfaces utilisateur statiques web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mes exercices de formation à LaPlateforme, j'ai développé des interfaces utilisateur statiques en HTML et CSS purs, sans framework. L'objectif était de maîtriser les fondamentaux du développement web avant d'utiliser des bibliothèques comme React.

Structure HTML sémantique :

J'ai structuré mes pages en utilisant les balises HTML sémantiques : <header>, <nav>, <main>, <section>, <article>, <footer>. L'utilisation de balises sémantiques améliore l'accessibilité (lecteurs d'écran) et le référencement naturel (SEO). J'ai également créé des formulaires complets avec les balises <form>, <input>, <label>, <select>, <textarea> et <button>, en respectant les bonnes pratiques d'accessibilité

```
<form class="contact-form">
  <div class="form-group">
    <label for="name">Nom :</label>
    <input type="text" id="name" name="name" required>
  </div>

  <div class="form-group">
    <label for="email">Email :</label>
    <input type="email" id="email" name="email" required>
  </div>

  <div class="form-group">
    <label for="subject">Sujet :</label>
    <select id="subject" name="subject">
      <option value="general">Question générale</option>
      <option value="theory">Théorie sur la série</option>
      <option value="fanart">Partage de fan art</option>
      <option value="spoiler">Discussion spoilers</option>
      <option value="other">Autre</option>
    </select>
  </div>

  <div class="form-group">
    <label for="message">Message :</label>
    <textarea id="message" name="message" placeholder="Partagez vos pensées sur The Flash..."
      required></textarea>
    <button type="submit" class="btn-submit">Envoyer le message</button>
  </div>
</form>
```

(attributs for/id, required, type approprié).

Mise en page CSS :

J'ai utilisé Flexbox et CSS Grid pour organiser les éléments sur la page. J'ai appliqué des propriétés avancées : dégradés (linear-gradient), border-radius, box-shadow, transitions CSS pour les effets au survol, et des variables CSS pour les couleurs du thème. Le CSS est organisé de façon modulaire, avec des classes réutilisables.

```
.flash-theme .contact-form {
  background: linear-gradient(135deg, rgba(255, 107, 107, 0.1), rgba(254, 202, 87, 0.1));
  padding: 2rem;
  border-radius: 15px;
  border: 2px solid rgba(255, 107, 107, 0.2);
  margin-bottom: 2rem;
}

.flash-theme .form-group select {
  width: 100%;
  padding: 0.75rem;
  border: 2px solid #ddd;
  border-radius: 5px;
  font-size: 1rem;
  transition: border-color 0.3s ease;
  background: white;
}
```

Media queries et responsive design :

J'ai rendu mes interfaces adaptatives pour tous les écrans grâce aux media queries CSS. J'ai défini plusieurs breakpoints :

- @media (max-width: 768px) → adaptation tablette : réduction des paddings, réorganisation des colonnes
- @media (max-width: 480px) → adaptation mobile : réduction des tailles de police, mise en colonne unique

```
@media (max-width: 768px) {
  .flash-theme .achievements {
    padding: 1rem;
  }

  .flash-theme .contact-form {
    padding: 1.5rem;
  }

  .flash-theme .fansite-header h1 {
    font-size: 2.5rem;
  }
}

@media (max-width: 480px) {
  .flash-theme .fansite-header h1 {
    font-size: 2rem;
  }

  .flash-theme .content-section h2::after {
    font-size: 1.2rem;
  }
}
```

Cela permet au site de s'afficher correctement sur desktop, tablette et smartphone sans changer le contenu HTML.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)

HTML5 (balises sémantiques, formulaires), CSS3 (Flexbox, Grid, transitions, dégradés, variables CSS), media queries pour le responsive design, Visual Studio Code avec l'extension Prettier.

3. Avec qui avez-vous travaillé ?

Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme, sous la supervision des formateurs.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Exercices de formation – Intégration HTML/CSS

Période d'exercice ► Du 23/06/2025 au 27/06/2025

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

Activité-type

1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°4 ▶

Développer la partie dynamique des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mes exercices de formation, j'ai développé des fonctionnalités dynamiques en JavaScript vanilla, c'est-à-dire sans framework. L'objectif était de comprendre les mécanismes fondamentaux de l'interactivité web avant d'utiliser des outils plus abstraits.

Les écouteurs d'événements (addEventListener) :

Un écouteur d'événement est une fonction qui "écoute" une action de l'utilisateur (clic, survol, scroll, saisie clavier...) sur un élément HTML et déclenche une action en réponse. La syntaxe est : `element.addEventListener('événement', callbackFunction)`.

J'ai utilisé `addEventListener` pour rendre mes interfaces réactives aux interactions utilisateur. Par exemple, sur un carousel de projets, j'ai écouté l'événement 'wheel' (molette de la souris) pour faire défiler les éléments horizontalement. La fonction callback reçoit l'objet événement (e) qui contient des informations sur l'action (e.deltaY pour la direction du scroll, e.preventDefault() pour bloquer le comportement par défaut du navigateur).

```
carouselContainer.addEventListener('wheel', (e) => {
  e.preventDefault();
  const itemWidth = carouselContainer.querySelector('div').offsetWidth +
24;  const visibleItems = Math.floor(carouselWrapper.offsetWidth / itemWidth);

  if (e.deltaY > 0 && currentIndex < totalItems - visibleItems) {
    currentIndex++;
    updateCarousel();
  } else if (e.deltaY < 0 && currentIndex > 0) {
    currentIndex--;
    updateCarousel();
  }
});
```

Le burger button (menu hamburger) :

Le menu burger est un bouton composé de 3 lignes horizontales qui apparaît sur mobile et permet d'ouvrir/fermer le menu de navigation. Son fonctionnement repose sur JavaScript :

1. On sélectionne le bouton et le menu via `document.querySelector()`
2. On ajoute un écouteur sur l'événement 'click' du bouton
3. À chaque clic, on utilise `classList.toggle('active')` pour ajouter ou retirer une classe CSS sur le menu
4. En CSS, la classe `.active` passe le menu de `display:none` à `display:flex`, avec une transition pour l'animation d'ouverture

Manipulation du DOM :

Le DOM (Document Object Model) est la représentation en mémoire de la page HTML. JavaScript permet de le manipuler en temps réel : modifier le texte d'un élément (`textContent`), changer ses classes (`classList`), créer ou supprimer des éléments (`createElement`, `appendChild`, `removeChild`), ou modifier ses attributs.

Frameworks utilisés par la suite :

Après avoir maîtrisé JavaScript vanilla, j'ai utilisé React (framework front-end) et Three.js (bibliothèque 3D) dans mes projets plus avancés. React permet de structurer une application en composants réutilisables et de gérer l'état de l'interface de façon déclarative. Three.js est une bibliothèque JavaScript permettant de créer des scènes 3D dans le navigateur via WebGL.

2. Précisez les moyens utilisés :

JavaScript ES6+ (vanilla), `addEventListener`, manipulation du DOM (`querySelector`, `classList`, `createElement`), Visual Studio Code. React.js et Three.js utilisés en complément comme frameworks/bibliothèques sur les projets suivants.

3. Avec qui avez-vous travaillé ?

DOSSIER PROFESSIONNEL (DP)

Projet personnel, développé en totale autonomie.

4. Contexte

Nom de l'entreprise, organisme ou association ▶		LaPlateforme (centre de formation)
Chantier, atelier, service	▶	Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme.
Période d'exercice	▶	Du 07/07/2025 au 18/07/2025

5. Informations complémentaires (facultatif)

Activité-type

2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°5 ▶

Mettre en place une base de données relationnelle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur le projet MarsAI, j'ai conçu et mis en place la base de données relationnelle MySQL. J'ai d'abord réalisé le modèle conceptuel de données (MCD) puis le modèle logique (MLD), aboutissant à une dizaine de tables interconnectées.

Scripts de création des tables (CREATE TABLE) :

J'ai rédigé les scripts SQL de création des tables en définissant pour chaque colonne son type (INT, VARCHAR, TEXT, DATETIME, TINYINT), ses contraintes (NOT NULL, DEFAULT, UNIQUE) et en ajoutant des COMMENT pour documenter les champs. Chaque table possède une clé primaire (PRIMARY KEY) sur le champ id avec AUTO_INCREMENT, ce qui garantit l'unicité de chaque enregistrement.

```
DROP TABLE IF EXISTS `users`;
CREATE TABLE `users` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  `profile_picture` varchar(500) DEFAULT NULL COMMENT 'URL photo de profil Scaleway (obligatoire pour jury)',
  `profile_picture_key` varchar(500) DEFAULT NULL COMMENT 'Clé objet Scaleway pour suppression',
  `role_title` varchar(255) DEFAULT NULL COMMENT 'Fonction du jury (ex: CHERCHEUSE IA / OPENAI)',
  `bio` text DEFAULT NULL COMMENT 'Description du membre du jury (optionnel)',
  `must_reset_password` tinyint(1) NOT NULL DEFAULT 0 COMMENT '1 = doit changer son mot de passe au prochain login (jury 1ère connexion)',
  PRIMARY KEY (`id`),
  UNIQUE KEY `email` (`email`),
  KEY `idx_users_jury` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Clés primaires et clés étrangères :

Les clés primaires (PRIMARY KEY) identifient de façon unique chaque ligne d'une table. Les clés étrangères (FOREIGN KEY) créent des liens entre les tables et garantissent l'intégrité référentielle : on ne peut pas associer un film à un utilisateur qui n'existe pas. J'ai également utilisé des index (KEY) pour optimiser les performances des requêtes fréquentes.

Scripts d'insertion de données de test (INSERT INTO) :

J'ai rédigé des scripts INSERT INTO pour peupler les tables avec des données de test réalistes. Les mots de passe sont systématiquement hashés avec bcrypt avant insertion (jamais en clair dans la base).

```
INSERT INTO `users` (`id`, `name`, `email`, `password`, `created_at`, `must_reset_password`) VALUES
(1, 'Admin MarsAI', 'admin@marsai.com', '$2a$10$8K1p/a0dL1LXMIgoEDFrwOjL8N5M1R1qH0gL1V7mF3q.d3X505Ixe',
NOW(), 0);
```

Création d'un utilisateur MySQL dédié (pas root) :

Par sécurité, j'ai créé un utilisateur MySQL dédié au projet avec des droits limités (SELECT, INSERT, UPDATE, DELETE) sur la base de données marsai uniquement. Utiliser root en production est une mauvaise pratique : si l'application est compromise, l'attaquant aurait accès à toutes les bases de données du serveur. Avec un utilisateur dédié, les droits sont limités au strict nécessaire.



```
CREATE USER 'Antonin'@'localhost' IDENTIFIED BY 'MotDePasseSuperSur';  
GRANT SELECT, INSERT, UPDATE, DELETE ON marsai.* TO  
Antonin@localhost;
```

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. La conception de la base de données a été réalisée collectivement lors de sessions de travail en équipe.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Projet MarsAI – Module base de données

Période d'exercice ► Du 19/01/2026 au 23/01/2026

5. Informations complémentaires (facultatif)

Activité-type

2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°6 ▶

Développer des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur le projet MarsAI, j'ai développé les composants d'accès aux données en adoptant une architecture en couches inspirée de Java Spring Boot, avec une couche repository dédiée aux requêtes SQL.

La couche repository :

La couche repository est une couche d'abstraction dont le seul rôle est d'effectuer les appels à la base de données. Elle contient toutes les requêtes SQL du projet et ne contient aucune logique métier. Ce découpage respecte le principe de séparation des responsabilités : si on change de base de données demain, on ne modifie que le repository, pas le reste du code.

Les requêtes préparées :

Toutes mes requêtes SQL utilisent des requêtes préparées avec des paramètres nommés (:param). Une requête préparée sépare le code SQL des données : les valeurs de l'utilisateur sont toujours transmises comme paramètres liés, jamais concaténées dans la chaîne SQL. Cela protège efficacement contre les injections SQL, qui sont l'une des vulnérabilités les plus courantes des applications web.

ORM Sequelize et SQL manuel :

J'ai utilisé Sequelize comme ORM (Object-Relational Mapper) pour la définition des modèles et la gestion des migrations. Cependant, toutes les requêtes SQL sont écrites manuellement via `sequelize.query()` avec

`QueryTypes.SELECT/INSERT/UPDATE/DELETE`. Cela me

permet de démontrer ma maîtrise du SQL tout en bénéficiant de la structure de Sequelize pour les modèles.

Requête principale – Récupération des courts-métrages avec filtres :

J'ai développé la requête `getFilmsWithFilters` qui récupère les courts-métrages depuis la base avec des filtres dynamiques (statut, catégorie, tri, pagination). La requête utilise des `LEFT JOIN` pour récupérer les catégories associées, `GROUP_CONCAT` pour regrouper les catégories multiples, `COUNT(*) OVER()` pour la pagination, et des paramètres nommés (:status, :limit, :offset) pour les filtres.

Accès aux données NoSQL – Portfolio FiveM (MongoDB) :

En parallèle, sur mon projet personnel portfolio (antonin-tacchi.com), j'ai développé les composants d'accès aux données en NoSQL avec MongoDB et Mongoose. Contrairement à une base relationnelle, MongoDB stocke les données sous forme de documents JSON imbriqués, ce qui correspond bien aux données de

```
export const getFilmsWithFilters = async ({
  status = null,
  limit = 20,
  offset = 0,
  sortField = "created_at",
  sortOrder = "DESC",
}) => {
  const allowedSortFields = new Set(["created_at", "title", "country", "id"]);
  const safeField = allowedSortFields.has(sortField) ? sortField :
    "created_at";
  const safeSortOrder = String(sortOrder).toLowerCase() === "asc" ? "ASC" : "DESC";
  const safeLimit = Math.min(50, Math.max(1, parseInt(limit, 10) || 20));
  const safeOffset = Math.max(0, parseInt(offset, 10) || 0);

  const rows = await sequelize.query(
    `SELECT
      f.id,
      f.title,
      f.title_english,
      f.country,
      f.description,
      f.description_english,
      f.poster_url,
      f.thumbnail_url,
      f.youtube_url,
      f.director_firstname,
      f.director_lastname,
      f.created_at,
      f.ai_tools_used,
      f.status,
      f.classification,
      f.ai_certification,
      f.rejection_reason,
      GROUP_CONCAT(c.name SEPARATOR ', ' ) AS categories,
      COUNT(*) OVER() AS total
    FROM films f
    LEFT JOIN film_categories fc ON f.id = fc.film_id
    LEFT JOIN categories c ON fc.category_id = c.id
    WHERE (:status IS NULL OR f.status = :status)
    ORDER BY f.${safeField} ${safeSortOrder}
    LIMIT :limit OFFSET :offset`,
    {
      replacements: { status, limit: safeLimit, offset: safeOffset },
      type: QueryTypes.SELECT,
    }
  );
};
```

projets ayant des attributs variables (titre, description, images, technologies utilisées, lien en ligne).

J'ai défini un schéma Mongoose pour les projets, conçu les routes CRUD de l'API REST (GET /projects, POST /projects, PUT /projects/:id, DELETE /projects/:id), et utilisé les méthodes Mongoose (find, findById, save, findByIdAndUpdate, findByIdAndDelete) pour interagir avec la base MongoDB Atlas. Cette expérience m'a permis de comprendre les différences fondamentales entre SQL et NoSQL : flexibilité du schéma, requêtes orientées document, et adaptées à des données semi-structurées.

2. Précisez les moyens utilisés :

Node.js, Express.js (framework web), Sequelize ORM (modèles et migrations uniquement), sequelize.query() avec QueryTypes pour les requêtes SQL manuelles, MySQL 8.0, architecture en couches (repository, service, controller, dto)

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. J'étais le référent technique sur la partie back-end et architecture.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► **Projet MarsAI – Couche repository et accès données**

Période d'exercice ► Du 26/01/2026 au 14/02/2026

5. Informations complémentaires (facultatif)

Activité-type

2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°7 ► Développer des composants métier coté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur le projet MarsAI, j'ai développé les composants métier côté serveur dans la couche service. Un composant métier regroupe l'ensemble des règles et traitements propres à une fonctionnalité de l'application, indépendamment de la couche HTTP (controller) et de la couche données (repository).

Composant métier – Inscription d'un utilisateur (register) :

Le service d'inscription illustre bien ce qu'est un vrai composant métier car il enchaîne plusieurs règles :

1. Vérification de l'unicité de l'email : appel à

`userRepository.emailExists(email)` pour s'assurer qu'aucun compte n'existe déjà avec cet email. Si l'email existe, une erreur avec statut HTTP 400 est levée.

2. Hashage du mot de passe : avant toute insertion en base, le mot de passe est hashé avec `bcrypt` (facteur de coût 10). Le mot de passe en clair n'est jamais stocké.

3. Création du compte : appel à `userRepository.create()` avec les données validées et le mot de passe hashé.

4. Attribution du rôle par défaut : appel à `userRepository.assignRole(user.id, 1)` pour attribuer le rôle 'user' au nouveau compte.

5. Récupération des rôles et génération du token JWT : le token JWT est signé avec les informations de l'utilisateur (id, email, rôles) et la clé secrète stockée dans les variables d'environnement.

6. Retour sécurisé : le mot de passe est retiré de l'objet retourné (destructuring) avant de renvoyer les données au controller. L'utilisateur ne reçoit jamais son mot de passe hashé dans la réponse.

Rôle du JWT dans ce composant :

Le JWT (JSON Web Token) est ici un outil au service de la logique d'inscription : il permet à l'utilisateur d'être immédiatement authentifié après son inscription sans avoir à se reconnecter. Le vrai composant métier est la logique d'inscription avec toutes ses règles, pas le JWT lui-même.

```
export const register = async ({ email, password, name }) => {
  const exists = await userRepository.emailExists(email);
  if (exists) throw Object.assign(new Error("Un utilisateur avec cet email existe déjà"), { status: 400 });

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await userRepository.create({ email, password: hashedPassword, name });

  await userRepository.assignRole(user.id, 1);

  const rolesRaw = await userRepository.getRoleIds(user.id);
  const roles = normalizeRoleIds(rolesRaw);

  const token = jwt.sign({ userId: user.id, email: user.email, roles }, SECRET, { expiresIn: JWT_EXPIRES_IN });

  const { password: _pw, ...userWithoutPassword } = user;
  return { user: userWithoutPassword, token };
};
```

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)

Node.js, Express.js, architecture en couches (service → repository), bcrypt pour le hashage des mots de passe, jsonwebtoken (JWT) pour la génération du token d'authentification, variables d'environnement (JWT_SECRET, JWT_EXPIRES_IN).

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. J'étais le référent technique sur la partie back-end et j'ai développé l'ensemble des services métier.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

DOSSIER PROFESSIONNEL (DP)

Chantier, atelier, service	► Projet MarsAI – Composants métier back-end
Période d'exercice	► Du 16/02/2026 au 20/02/2026

5. Informations complémentaires *(facultatif)*

Activité-type

2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°8 ▶

Documenter le déploiement d'une application dynamique web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai documenté le déploiement complet de l'application MarsAI basée sur Docker. Cette documentation permet à n'importe quel membre de l'équipe de lancer l'environnement complet en quelques commandes.

Les variables d'environnement :

Une variable d'environnement est une valeur de configuration externe au code source, stockée dans un fichier .env. Elle permet de :

- Sécuriser les informations sensibles (mots de passe, clés secrètes) en les sortant du code
- Adapter l'application à différents environnements (développement, production) sans modifier le code
- Ne jamais committer les valeurs sensibles sur GitHub (le .env est dans .gitignore)

Dans Node.js, on accède aux variables via `process.env.NOM_VARIABLE`. J'ai créé un fichier `.env.exemple` (sans valeurs sensibles) pour documenter toutes les variables nécessaires au projet.

Docker et Docker Compose :

J'ai rédigé le fichier `docker-compose.yml` qui orchestre les trois services de l'application : le front-end React, le back-end Node.js/Express et la base de données MySQL. Pour chaque service, j'ai configuré :

- `build` : le chemin vers le Dockerfile du service
- `environment` : les variables d'environnement injectées dans le conteneur (`DB_HOST`, `DB_USER`, `JWT_SECRET`...)
- `env_file` : le fichier `.env` local utilisé comme source de variables

```
backend:
  build:
    context: ./back-end
    dockerfile: Dockerfile
  container_name: marsai-backend
  environment:
    CORS_ORIGIN: "${CORS_ORIGIN:-http://localhost:5173,http://127.0.0.1:5173}"
    DB_HOST: mysql
    DB_PORT: "3306"
    DB_USER: "${DB_USER}"
    DB_PASSWORD: "${DB_PASSWORD}"
    DB_NAME: marsai
    JWT_SECRET: "${JWT_SECRET}"
  ports:
    - "${BACKEND_PORT}:${BACKEND_PORT}"
  env_file:
    - ./back-end/.env
  depends_on:
    mysql:
      condition: service_healthy
  restart: unless-stopped
```

- depends_on : l'ordre de démarrage des services (le back attend que MySQL soit healthy avant de démarrer)
- restart: unless-stopped : redémarrage automatique en cas de crash

README et documentation de déploiement :

J'ai rédigé un fichier README.md complet expliquant les prérequis (Docker Desktop installé), les étapes de lancement (git clone → copier .env.exemple en .env → remplir les valeurs → docker-compose up --build), les commandes utiles (docker-compose down, docker logs, docker exec) et l'architecture du projet.

2. Précisez les moyens utilisés :

Docker Desktop, Docker Compose (docker-compose.yml), Dockerfile pour chaque service, fichiers .env et .env.exemple, Markdown pour la rédaction du README, Git/GitHub pour partager la documentation.

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. La documentation a été rédigée par mes soins en tant que coordinateur technique du projet.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Projet MarsAI – Infrastructure Docker et déploiement

Période d'exercice ► Du 09/03/2026 au 18/03/2026

5. Informations complémentaires (facultatif)

tes, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

DOSSIER PROFESSIONNEL (DP)



Cliquez ici.

Cliquez ici pour taper du texte.

Cliquez ici pour
sélectionner une date.

DOSSIER PROFESSIONNEL (DP)

Déclaration sur l'honneur

Je soussigné(e) Antonin TACCHI,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à Martigues le 07/05/2026
pour faire valoir ce que de droit.

Signature :

DOSSIER PROFESSIONNEL (DP)

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé

Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

ANNEXES

(Si le RC le prévoit)

